

TRAIVO ONE — Implementationsrapport: Tre Identifierade Gap

Datum: 2026-06-10

Status: Minimal gap-analys mot befintlig mogen plattform

Arkitektur: ADR v3, metadatasystem, OBJ-NNN-numrering

INLEDNING

Traivo One är en **mogen webbplattform** med **180 databastabeller** och **101 färdiga webbsidor**. Plattformen har en väletablerad arkitektur med metadatastyrt UI, hierarkisk objektmodell (OBJ-NNN-numrering) och fullständig CRUD för hela affärsflödet.

Denna rapport är **inte en greenfield-plan**. Den adresserar tre specifika, avgränsade gap som identifierats genom granskning av befintlig kodbas. Allt annat — kundhantering, objekthantering, ordrar, fakturering, kalenderplanering, klusterhantering, m.m. — finns redan implementerat och fungerar.

Tack till Replit för den noggranna korrigeringen av den ursprungliga analysen. Den säkerställde att vi arbetar mot verkligheten, inte mot en felaktig bild av systemet.

GAP 1: “SNÖRET” — Processflödet på Kundnivå

Nuläge

- **Pipeline-visualisering finns** redan för kluster (cluster-nivå).
- **CustomerDetailPage** visar kundstatistik, kontaktinformation och relaterade objekt — men saknar en sammanhållen processflödesvy.
- Alla underliggande data finns i befintliga tabeller: objekt, abonnemang, ordrar, utförda jobb och fakturor är redan kopplade via relationer.
- Användaren måste idag navigera mellan flera sidor för att förstå en kunds hela livscykel.

Önskad funktionalitet

En **dedikerad processflödesvy per kund** som visualiserar hela “snöret”:

Objekt → Abonnemang → Aktiva ordrar → Utförda → Fakturerade

Krav:

- Horisontell pipeline-visualisering (steg-för-steg, liknande en Kanban/funnel)
- Varje steg visar antal och summor
- Klickbara steg som expanderar till detaljlista
- Markering av flaskhalsar (t.ex. utförda men ej fakturerade)
- Filtrering per tidsperiod

Implementation

Komponenter som utökas

- **CustomerDetailPage** — lägg till en ny flik/sektion `Processflöde`
- Alternativt: ny route `/customers/:id/pipeline`

Befintliga tabeller som används

Steg	Tabell(er)	Nyckelrelation
Objekt	<code>objects</code>	<code>customer_id</code>
Abonnemang	<code>subscriptions</code>	<code>object_id</code> → <code>customer_id</code>
Aktiva ordrar	<code>orders</code> (status: active)	<code>subscription_id</code>
Utförda	<code>orders</code> (status: completed)	<code>subscription_id</code>
Fakturerade	<code>invoices</code>	<code>order_id</code>

UI-komponenter

CustomerPipelineView (ny komponent)

- [-] PipelineStepCard ☒ 5 (återanvändbar)
- [-] StepHeader (ikon, namn, antal, summa)
- [-] StepProgressBar (% av föregående steg)
- [-] StepDetailDrawer (expanderbar lista)
- [-] PipelineFilterBar (tidsperiod, objekttyp)
- [-] PipelineBottleneckAlert (varningar)

API-endpoints

Sannolikt behövs **ett nytt aggregerings-endpoint**:

```
GET /api/customers/:id/pipeline-summary
```

Response:

```
{
  "objects": { "count": 12, "value": null },
  "subscriptions": { "count": 8, "active": 6, "value": 45000 },
  "activeOrders": { "count": 4, "value": 12000 },
  "completedOrders": { "count": 18, "value": 54000 },
  "invoiced": { "count": 15, "value": 48000, "unpaid": 6000 }
}
```

Alternativt kan detta byggas som en **vy/query i befintligt metadatasystem** om arkitekturen stödjer aggregerade vyer.

Estimerad komplexitet

Del	Uppskattning
Backend-endpoint (aggregering)	0,5–1 dag
PipelineStepCard-komponent	0,5 dag
CustomerPipelineView (sammansättning)	1 dag
Filtrering + bottleneck-logik	0,5 dag
Totalt	2,5–3 dagar

Risk: Låg. Alla data finns redan; detta är en ren presentationsförändring med ett aggregeringsend-point.

GAP 2: Zoombar Kalender-tidslinje

Nuläge

- **AnnualPlanningPage** finns — visar årsöversikt.
- **WeekPlanner** finns — visar veckoplanering med detaljerade tidsslottar.
- Dessa är **separata sidor** utan övergång mellan dem.
- Användaren måste manuellt navigera mellan årsvy och veckovy.
- Det saknas en mellanliggande kvartal- och månadsvy.

Önskad funktionalitet

En **sammanhållen zoombar tidslinje** med fem nivåer:

År → Kvartal → Månad → Vecka → Dag

Krav:

- Sömlös zoom mellan nivåer (scroll/pinch eller knappar)
- Kontextuell zoom: klick på en månad i årsvy zoonar in till den månaden
- Varje nivå visar relevant densitet av data (ordrar, bokningar, resurser)
- Behåll befintlig funktionalitet i AnnualPlanningPage och WeekPlanner
- Breadcrumb-navigation: 2026 > Q2 > Juni > Vecka 24 > Tisdag

Implementation

Strategi: Wrapper runt befintliga komponenter

Bygg **inte** om AnnualPlanningPage eller WeekPlanner. Skapa istället en ny wrapper-komponent som orkestrerar zoom-nivåer och återanvänder befintliga vyer för ändnivåerna.

Ny komponentstruktur

```
ZoomableTimelinePage (ny route: /planning/timeline)
├── TimelineZoomControls
│   ├── ZoomLevelSelector (År/Kvartal/Månad/Vecka/Dag)
│   ├── TimelineBreadcrumb (navigationskedja)
│   └── ZoomButtons (+/- eller slider)
├── TimelineViewport (renderar rätt vy baserat på zoom)
│   ├── YearView (återanvänd/adapttera AnnualPlanningPage)
│   ├── QuarterView (ny 3 månader i grid)
│   ├── MonthView (ny dag-för-dag med bokningar)
│   ├── WeekView (återanvänd/adapttera WeekPlanner)
│   └── DayView (ny timplottar med detaljer)
└── TimelineContext (state management)
```

State management

```
interface TimelineState {
  zoomLevel: 'year' | 'quarter' | 'month' | 'week' | 'day';
  focusDate: Date;           // centrumdag för vyn
  dateRange: [Date, Date];  // synligt intervall
  filters: {
    resourceIds?: string[];
    orderTypes?: string[];
  };
}

// Zoom-övergångar
function zoomIn(state: TimelineState, targetDate: Date): TimelineState;
function zoomOut(state: TimelineState): TimelineState;
```

Befintliga komponenter att återanvända

Zoom-nivå	Befintlig komponent	Anpassning
År	AnnualPlanningPage	Extrahera renderingslogik till delkomponent
Kvartal	—	Ny, men baserad på Annual-PlanningPage-layout
Månad	—	Ny, standardkalender-grid med bokningsdata
Vecka	WeekPlanner	Extrahera renderingslogik till delkomponent
Dag	WeekPlanner (1 dag)	Filtrera WeekPlanner till en dag

API-endpoints

Befintliga endpoints bör redan leverera data för kalendervisning. Eventuellt behövs:

```
GET /api/planning/events?from=2026-01-01&to=2026-12-31&granularity=month
```

Parametern `granularity` styr aggregeringsnivå för att undvika att ladda tusentals enskilda händelser i årsvy.

Ny route

```
/planning/timeline → ZoomableTimelinePage
/planning/timeline?zoom=month&date=2026-06 → Direkt till månadsvy
```

Behåll befintliga routes (`/planning/annual` , `/planning/week`) för bakåtkompatibilitet.

Estimerad komplexitet

Del	Uppskattning
TimelineState + zoom-logik	1 dag
ZoomControls + Breadcrumb	0,5 dag
QuarterView (ny)	1 dag
MonthView (ny)	1 dag
DayView (ny)	0,5 dag
Adaptera AnnualPlanningPage → YearView	1 dag
Adaptera WeekPlanner → WeekView	0,5 dag
Integration + övergångsanimationer	1 dag
Totalt	6-7 dagar

Risk: Medel. Refaktorering av befintliga komponenter kan avslöja hårdkodade antaganden. Rekommendation: börja med att extrahera AnnualPlanningPage och WeekPlanner till återanvändbara delkomponenter innan den nya wrappern byggs.

GAP 3: Redundansen Objekttyp = Hierarkinivå

Nuläge

- Systemet har två parallella begrepp: **objekttyp** och **hierarkinivå**.
- I praktiken speglar de ofta varandra, t.ex.:
- Objekttyp: "Rum" — Hierarkinivå: "Rum"
- Objekttyp: "Fastighet" — Hierarkinivå: "Fastighet"
- Detta skapar **UX-förvirring**: användaren ser samma begrepp dubblerat i formulär och listor.
- I OBJ-NNN-numreringssystemet kan hierarkinivå härledas från objektets position i trädet.

Önskad funktionalitet

- **Eliminera redundansen** genom att låta hierarkinivå definieras av objektets position i trädstrukturen, inte som ett separat fält.
- Objekttyp behålls som primärt begrepp (det ger semantisk mening: "Rum", "Byggnad", etc.).
- Hierarkinivå blir **beräknad**, inte lagrad — baserat på träddjup.

Implementation

Analys: Är fälten verkligen redundanta?

Innan implementation — verifiera:

1. Finns det fall där objekttyp $\neq$ hierarkinivå? (T.ex. en "Rum" på nivå 2 OCH nivå 3?)
2. Används hierarkinivå i affärslogik (prissättning, filtrering, rapporter)?
3. Finns det API-konsumenter som förlitar sig på hierarkinivå-fältet?

Om svaret är "nej" på alla: Säker att ta bort.

Om svaret är "ja" på någon: Överväg att dölja i UI men behålla i datamodell.

Påverkade komponenter

Komponent	Förändring
Objektformulär (create/edit)	Ta bort hierarkinivå-dropdown
Objektlistor	Ta bort hierarkinivå-kolumn eller ersätt med beräknat träddjup
Filterkomponenter	Slå ihop duplicerade filter till ett
Metadatakonfiguration	Ta bort eller deprecera hierarkinivå som konfigurerbart fält
Rapporter/exporter	Ersätt hierarkinivå med beräknat värde

Datamodelländringar

Alternativ A — Mjuk deprecering (rekommenderat):

```
-- Behåll kolumnen men sluta visa den i UI
-- Lägg till beräknat fält i vy
CREATE OR REPLACE VIEW objects_with_level AS
SELECT o.*,
       (SELECT COUNT(*) FROM object_hierarchy h
        WHERE h.descendant_id = o.id AND h.ancestor_id != o.id) as tree_depth
FROM objects o;
```

Alternativ B — Hård borttagning:

```
-- Migrera bort kolumnen (VARNING: breaking change)
ALTER TABLE objects DROP COLUMN hierarchy_level;
```

Rekommendation: Alternativ A. Det ger bakåtkompatibilitet och låter migrationen ske gradvis.

UI/UX-städning

1. **Formulär:** Ta bort hierarkinivå-fältet. Objekttyp räcker.
2. **Träddvy:** Visa träddjup visuellt genom indentation (redan troligen fallet).
3. **Filterbar:** Slå ihop "Filtrera på objekttyp" och "Filtrera på hierarkinivå" till ett filter.
4. **Tooltips:** Lägg till tooltip som visar träddjup om användaren behöver den informationen.

Migration

1. Verifiera att ingen affärslogik refererar till `hierarchy_level` direkt.
2. Skapa beräknad vy (Alternativ A).
3. Uppdatera UI-komponenter att läsa från beräknat fält.
4. Ta bort `hierarchy_level` från formulär.
5. Behåll kolumnen i databasen i minst en release-cykel.
6. Ta bort kolumnen i framtida release efter verifiering.

Estimerad komplexitet

Del	Uppskattning
Analys och verifiering av beroenden	0,5 dag
Beräknad vy/fält	0,5 dag
UI-borttagning (formulär, listor, filter)	1 dag
Tester och regression	0,5 dag
Totalt	2-3 dagar

Risk: Låg-Medel. Huvudrisken är okända beroenden i affärslogik eller rapporter. Analys-steget (dag 1) avgör om detta är trivialt eller kräver mer försiktighet.

SAMMANFATTNING

Total estimerad arbetsinsats

Gap	Beskrivning	Estimat
Gap 1	Snöret — kundnivå-pipeline	2,5-3 dagar
Gap 2	Zoombar kalender-tidslinje	6-7 dagar
Gap 3	Objekttyp/hierarkinivå-redundans	2-3 dagar
Totalt		11-13 dagar

Rekommenderad prioritering

Prioritet	Gap	Motivering
1	Gap 3 (Redundans)	Lägst risk, snabbast ROI, städar upp UX
2	Gap 1 (Snöret)	Hög affärsnytta, låg teknisk risk
3	Gap 2 (Tidslinje)	Störst scope, kräver refaktoring

Gap 3 först eftersom det är en ren UX-förbättring utan nya features. Gap 1 därefter för att det ger omedelbart affärsvärde. Gap 2 sist eftersom det kräver mest arbete och bör byggas inkrementellt.

Risker och beroenden

Risk	Sannolikhet	Påverkan	Mitigation
Gap 3: Okända beroenden på <code>hierarchy_level</code>	Medel	Medel	Analysera först, mjuk deprecering
Gap 2: Hårdkodade antaganden i Annual-PlanningPage	Medel	Hög	Extrahera delkomponenter som separat steg
Gap 2: Prestandaproblem vid årsvy med mycket data	Låg	Medel	Aggregerings-endpoint med granularity-param
Gap 1: Aggregerings-endpoint kan bli långsam	Låg	Låg	Caching eller materialiserad vy

Nästa steg

1. **Vecka 1:** Implementera Gap 3 (redundans-städning). Börja med beroendeanalys.
2. **Vecka 2:** Implementera Gap 1 (pipeline-vy). Backend-endpoint + UI-komponent.
3. **Vecka 3-4:** Implementera Gap 2 (tidslinje). Börja med komponentextraktion, sedan wrapper.
4. **Löpande:** Code review, tester och UX-validering med användare efter varje gap.

Denna rapport respekterar Traivo Ones befintliga arkitektur (ADR v3, metadatasystem, OBJ-NNN-numrering) och föreslår minimala, avgränsade tillägg snarare än omskrivningar.